

Programmation Fonctionnelle

TD n° 6 : Récursivité terminale

1 Récursivité terminale sur les listes

Exercice 1 : On considère la fonction `listn` ci-dessous, qui étant donné un entier positif `n` renvoie une liste contenant tous les entiers de `n` à 0. Jusqu'à quelle valeur de `n` fonctionne-t-elle ? (La tester dans le toplevel.)

```
let rec listn n =  
  if n = 0 then [0]  
  else n :: (listn (n - 1))
```

Cette fonction n'est pas récursive terminale. Pourquoi ? Modifier son implémentation pour la rendre récursive terminale. Jusqu'à quelle valeur de `n` fonctionne-t-elle ?

Exercice 2 : On considère la fonction `length` ci-dessous qui calcule la longueur d'une liste.

```
let rec length l = match l with  
| [] -> 0  
| _ :: l' -> 1 + (length l')
```

Utiliser votre fonction `listn` de l'exercice précédent pour tester `length` dans le toplevel. Jusqu'à quelle taille de liste cela fonctionne ? Modifier l'implémentation de la fonction `length` pour la rendre récursive terminale. Vérifier que cela améliore la situation.

Exercice 3 : Écrire une fonction récursive terminale `rev_append` qui prend en paramètre deux listes `left` et `right` et qui retourne la concaténation de liste `left` à l'envers et de la liste `right`. Par exemple, un appel à `rev_append [1;2;3] [4;5;6]` doit renvoyer `[3;2;1;4;5;6]`.

Exercice 4 : Écrire une fonction `append` qui concatène les deux listes passées en paramètre en faisant appel à `rev_append`. Comparer son comportement avec celui de l'opérateur `@` sur des listes de grande taille.

Note : on pourra utiliser la fonction `List.rev` fournie par la bibliothèque standard, dont l'implémentation est récursive terminale.

2 Récursivité terminale sur les arbres

On considère maintenant le type d'arbres binaires défini ci-dessous.

```
type 'a tree =  
  | Nil  
  | Node of 'a * 'a tree * 'a tree
```

Exercice 5 : On rappelle que la *taille* d'un arbre est le nombre de nœuds (constructeur `Node`). En guise d'échauffement, écrire une fonction `size : 'a tree -> int` (pas forcément récursive terminale) qui calcule la taille d'un arbre.

Exercice 6 : On veut maintenant implémenter une version récursive terminale de la fonction `size`. Dans un premier temps, nous allons implémenter la fonction auxiliaire suivante :

```
size_aux : 'a tree list -> int -> int
```

qui prend en paramètre une liste d'arbres `l` et un entier `n` et qui renvoie la somme :

- des tailles des arbres dans la liste `l`,
- et de l'entier `n`

Par exemple, le test ci-dessous doit renvoyer 15 :

```
let t1 = Node(1, Nil, Nil) in
let t2 = Node(2, t1, t1) in
let t3 = Node(3, t2, t2) in
size_aux [t1; t2; t3] 4
```

Implémenter la fonction `size_aux` de façon récursive terminale.

Note : pour que `size_aux` soit récursive terminale, celle-ci ne doit pas utiliser la fonction `size` de l'exercice précédent.

Exercice 7 : Réimplémenter la fonction `size` en utilisant un appel à `size_aux`.

Exercice 8 : (Bonus) En vous inspirant des questions précédentes, écrire une fonction récursive terminale `height : 'a tree -> int` qui calcule la hauteur d'un arbre binaire.¹

3 Encore des listes

Comme dans les exercices précédents, l'objectif est d'implémenter les fonctions suivantes en utilisant la récursivité terminale (les implémentations naïves de ces fonctions ont été vues dans le TP 3). Toutes les fonctions auxiliaires utilisées doivent aussi être récursives terminales.

Note : la fonction `List.rev_append` de l'Exercice 3 pourra être très utile.

Exercice 9 : Écrire une fonction `insert` qui étant donnée une liste supposée triée pour l'ordre strictement croissant et un élément `x`, renvoie la liste obtenue en insérant `x` à la bonne place. Si `x` est déjà dans la liste, celle-ci sera renvoyée telle quelle.

Exemples :

```
insert 5 [1;3;8] = [1;3;5;8]
insert 'e' ['a';'c';'g'] = ['a';'c';'e';'g']
```

Exercice 10 : En utilisant la fonction `insert`, écrire une fonction `sort` permettant de trier une `'a list` quelconque par ordre croissant, en fusionnant les doublons.

Exemple :

```
sort [7;8;5;2;8] = [2;5;7;8]
```

Exercice 11 : Écrire les opérations d'union et intersection (`union_sorted`, `inter_sorted`) de deux listes triées.

Exemples :

```
union_sorted [1;3;5] [2;5;8] = [1;2;3;5;8]
inter_sorted [1;3;5] [2;5;8] = [5]
```

1. Un peu de lecture pour les plus curieu-x-ses : *Mesurer la hauteur d'un arbre*, Jean-Christophe Filliâtre.
<https://inria.hal.science/hal-02315541/file/main-pp.pdf>